Actividad 3

Implementación de las funciones diseñadas en la tarjeta de desarrollo

Índice

1. Introducción y objetivos 2
2. Creación del proyecto 3
3. Asignación de pines de la FPGA 5
4. Síntesis, implementación y generación del Bitstream 9
5. Configuración de la FPGA con el diseño de la Actividad 1 11
6. Configuración de la FPGA con el diseño de la Actividad 2 16
Referencias 18

Actividad 3

Implementación de las funciones diseñadas en la tarjeta de desarrollo

1.- Introducción y objetivos

En esta actividad se va a proceder a implementar en la tarjeta de desarrollo las funciones descritas en la Actividad 1 y la Actividad 2. Con esto, se completa la fase de diseño e implementación de circuitos digitales empleando la tecnología basada en FPGA-VHDL.

Para la realización de esta actividad, se crea un nuevo proyecto Actividad 3, en el que se importará los códigos VHDL de las actividades anteriores.

Los objetivos a alcanzar en esta práctica son:

- Importar ficheros a un nuevo proyecto
- Seleccionar los pines de la FPGA para los puertos de entrada y salida
- Implementar el diseño en la tarjeta de desarrollo
- Modificar el *top level* de un proyecto

2.- Creación del proyecto

Para la creación del proyecto de Vivado de esta actividad, deben seguirse los pasos descritos en el Apartado 3 de la Actividad 0. En este caso, el nombre del proyecto es *Actividad3* (todo junto).

En esta ocasión, en el apartado *Add Sources* del asistente para crear el nuevo proyecto, se debe clicar en *Add Files* (Figura 1), Con esto se abre una nueva ventana en la que incorporamos los ficheros VHDL de diseño y simulación al proyecto que se está creando.

Se debe incorporar los ficheros *funciones.vhd*, *funciones_tb.vhd*, *funciones2.vhd* y *funciones2_tb.vhd* de las actividades anteriores.

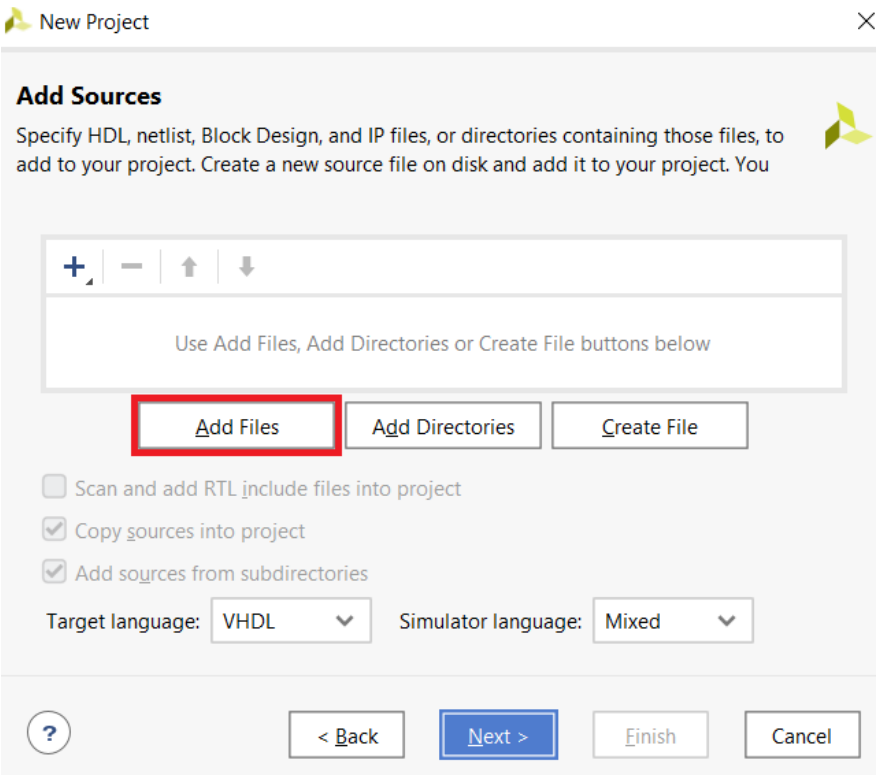

Figura 1. Captura de pantalla de *Add Sources*, con el botón *Add Files* resaltado en rojo.

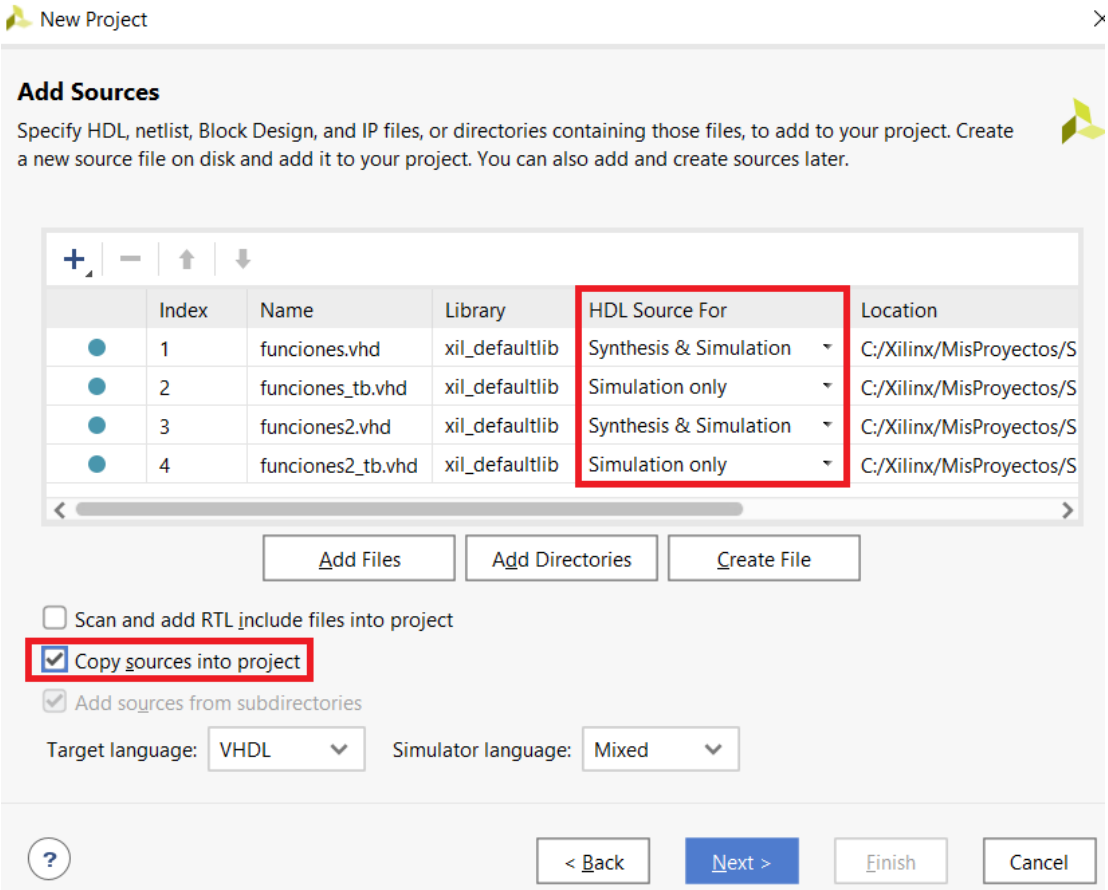

Figura 2. Captura de pantalla de *Add Sources*, con los ficheros a incorporar en el nuevo proyecto.

A medida que se van añadiendo los ficheros, es importante indicarle a Vivado si son códigos de diseño o de simulación en la columna *HDL Source for*. También es importante marcar la opción *Copy sources into project*. En la Figura 2 se muestra una captura la ventana del asistente con los ficheros añadidos y las opciones correctas seleccionadas resaltadas en rojo.

El proyecto se termina de crear seleccionando la FPGA que se va a emplear en el proyecto (ver Figura 10 de la Actividad 0). En la Figura 3 se muestra la jerarquía del proyecto recién creado. Puede apreciarse que en *Design Sources* aparecen los dos circuitos (en este caso funciones es el *top level*); y en *Simulation Sources* aparacen los dos *test bench* con sus circuitos bajo test asociados (en este caso *funciones2_tb* es el banco de pruebas activo en este momento para simulación). Tanto para *Design Sources* como para *Simulation Sources*, el *top level* puede modificarse aplicando el comando *Set as Top*. Para acceder a este comando, basta con clicar con el botón derecho del ratón sobre el circuito que se desea establecer como nuevo *top level*. El comando aparece en el menú emergente.

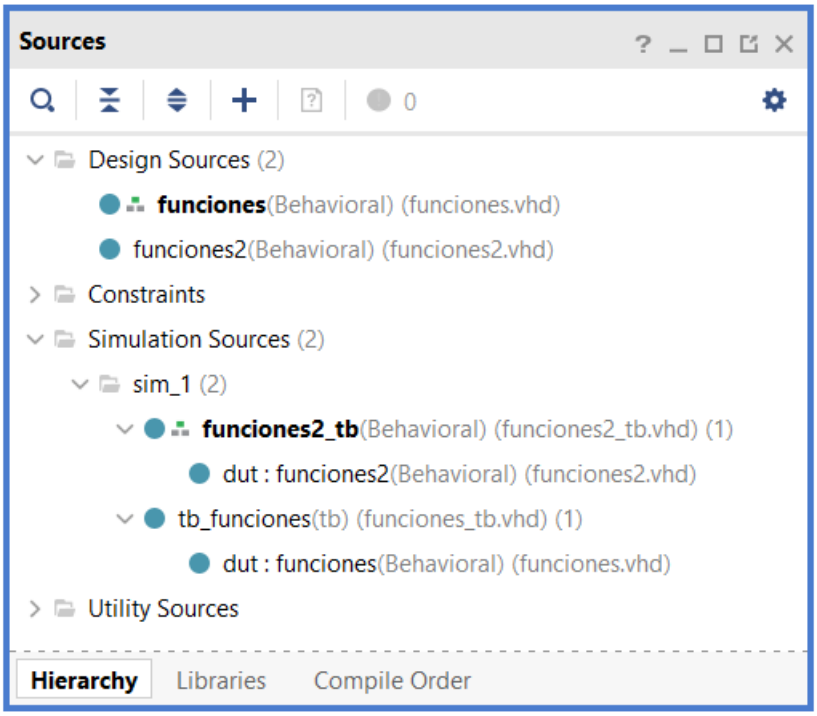

Figura 3. Captura de pantalla de la jerarquía del nuevo proyecto.

A partir de aquí, se puede volver a realizar la simulación de los dos circuitos (Apartado 7 de Actividad 1 y Apartado 5 de Actividad 2).

3.- Asignación de pines de la FPGA

Hasta ahora, la especificación de la FPGA era irrelevante, ya que en la Actividad 1 y la Actividad 2 únicamente se limitaban al diseño y simulación.

En las siguientes fases en el proceso de diseño es necesario indicar la FPGA y en qué pines de la FPGA asignamos todos y cada uno de los puertos de entrada y salida que tiene nuestro circuito digital a implementar. Esto se realiza usando restricciones de usuario (*user constrains* en inglés). Estas restricciones pueden ser temporales (especificar unos ciertos requerimientos de velocidad, por ejemplo) o de ubicación (en qué lugar físico de la FPGA implementaremos nuestro circuito lógico) o de asignación de pines (en qué pines físicos del dispositivo FPGA pondremos nuestros puertos de entrada y salida). Esto último es lo que nos interesa en este momento.

Cualquier tarjeta de desarrollo suele constar de una FPGA y ciertos periféricos conectados a ella que son útiles para comprobar en el propio circuito nuestro diseño. Estos periféricos suelen ser interruptores, botones, leds, displays, conectores serie, VGA y otros muchos. Así pues, interesa asignarles a los puertos de entrada y salida de nuestro diseño unos pines de la FPGA que estén conectados según a qué periféricos de la tarjeta.

Normalmente, se hace un análisis de los puertos de entrada/salida del circuito a implementar en FPGA para ver, dentro de la disponibilidad, qué periféricos serían los ideales para interactuar con el circuito.

El circuito asociado al código *funciones.vhd* se muestra en la Figura 4.

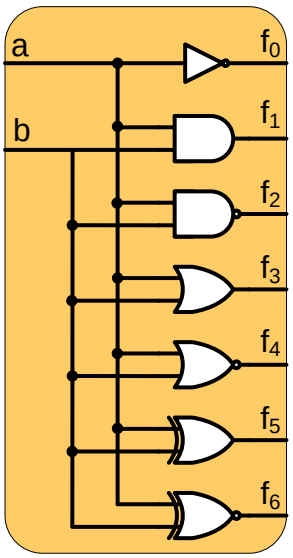

Figura 4. Circuito a implementar en la FPGA

En este caso, lo adecuado es asignar los puertos de entrada *a* y *b* a pines de la FPGA que estén conectados físicamente a interruptores, de tal manera que se puede controlar el valor lógico de las entradas al circuito (interruptor bajo, valor 0; e interruptor alto, valor 1). Para los puertos de salida, lo adecuado será asignarlos a los pines de la FPGA conectados a leds, de tal manera que si el valor de la función es 0 el led está apagado, y si es 1 el led está encendido.

Para la tarjeta de desarrollo Nexys A7, con la FPGA **XC7A100T-1CSG324C**, la información de la correspondencia entre los periféricos y los pines de la FPGA se puede encontrar:

- Serigrafiada en la propia tarjeta de desarrollo.
- En el documento de especificaciones de la tarjeta que provee el fabricante de la misma, la empresa Digilent [1].
- En el fichero de restricciones maestro que provee Digilent [2]

De cualquiera de las 3 maneras podemos encontrar la información que se muestra en la Tabla 1, donde se muestra los pines de conexión de la FPGA **XC7A100T-1CSG324C**, con los 16 interruptores y los 16 leds de la tarjeta de desarrollo Nexys A7. Para la implementación del circuito de la Figura 1, se han marcado en verde los pines que se van a emplear. Estos pines están conectados a los leds denominados LED[6], LED[5], LED[4], LED[3], LED[2], LED[1] y LED[0], y a los interruptores (*switches*) SW[1] y SW[0].

LED	15	14	13	12	11	10	9	8	7	6 f0	5 f1	4 f2	3 f3	2 f4	1 f5	0 f6
PIN	V11	V12	V14	V15	T16	U14	T15	V16	U16	U17	V17	R18	N14	J13	K15	H17
Sw.	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1 a	0 b
PIN	V10	U11	U12	H6	T13	R16	U8	t8	R13	U18	T18	R17	R15	M13	L16	J15

Tabla 1. Pines de la FPGA XC7A100T-1CSG324C conectados a los leds y *switches* de la tarjeta Nexys A7. En verde, los pines a usar de interfaz.

Para crear el fichero de restricciones de usar se procede de la siguiente manera:

1. Seleccionar *Add Sources*.
2. En la ventana emergente, seleccionar *Add or create constraints* (Figura 5)
3. En el siguiente paso, seleccionar *Add File* y añadir el fichero previamente descargado *Nexys-A7-100T-Master.xdc* [2]. Este fichero es la plantilla maestra con todos los pines de la FPGA categorizados por periféricos. La extensión *xdc* es el acrónimo de *Xilinx Design Constraints*.
4. Tras añadir la plantilla maestra, seleccionar *Finish* (Figura 6).

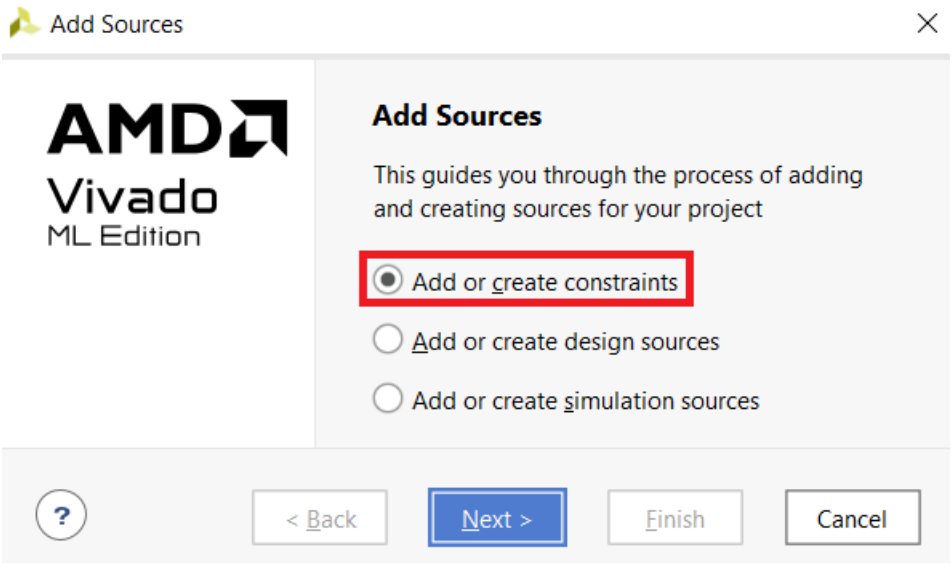

Figura 5. Captura de pantalla de *Add Sources* con *Add or create constraints* resaltado en rojo

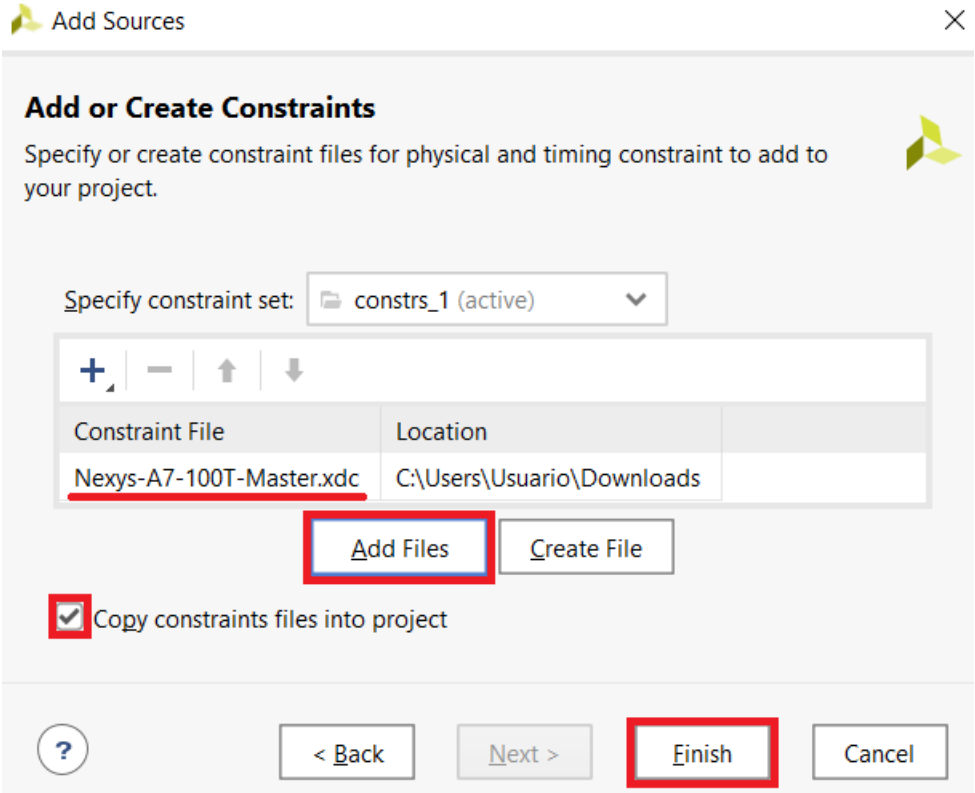

Figura 6. Captura de pantalla de *Add or create constraints* con la plantilla maestra subrayada resaltada en rojo

El archivo de asignaciones de pines añadido se incorpora en la ventana de jerarquías según se puede apreciar en la Figura 7.

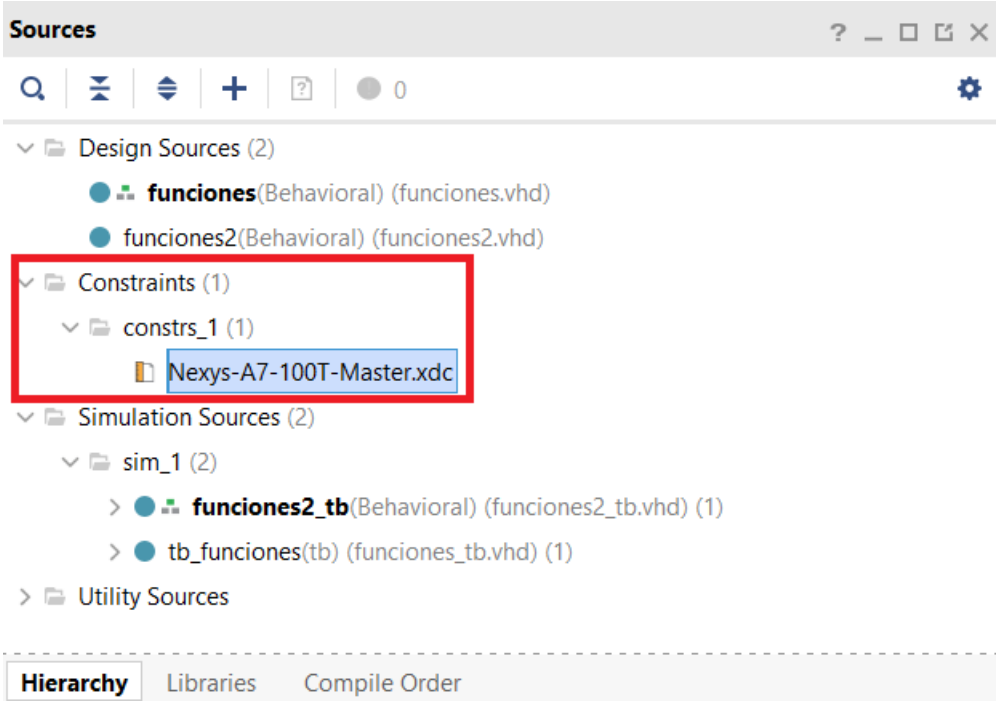

Figura 7. Captura de pantalla de la ventana de jerarquía con la plantilla maestra subrayada resaltada en rojo

La plantilla añadida al proyecto, *Nexys-A7-100T-Master.xdc*, contiene una cabecera de comentario en la que nos indica el modo de usarla para cada proyecto. Hay que proceder de la siguiente manera:

- Descomentar las líneas de código correspondientes a los pines que va a usar nuestro circuito (quitar #).
- Cambiar el nombre por defecto que tiene el puerto (en cada línea, justo después de *get_ports*), de acuerdo con el nombre de los puertos que tenga la entidad del módulo *top level*.

Teniendo en cuenta lo anterior, se descomentan las líneas 12 y 13 (correspondientes a la sección de *switches*, interruptores), poniendo los nombres *b* y *a* respectivamente, según se muestra en la Figura 8. Se puede contrastar los pines con la Tabla 1 (pines L16 y J15).

```
11 ##Switches
12 set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { b }];
13 set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { a }];
```

Figura 8. Captura de pantalla las líneas del archivo de restricciones correspondientes a los puertos de entrada *a* y *b* del circuito *funciones.vhd*.

Se procede de manera análoga con los pines correspondientes a los puertos de salida del diseño. En este caso, se descomentan las líneas de la 30 a 36, sustituyendo los nombres por *f6*, *f5*, *f4*, *f3*, *f2*, *f1*, *f0* (Figura 9). Se puede contrastar los pines con la Tabla 1 (H17, K15, J13, N14, R18, V17, U17).

```
29 ## LEDs
30 set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { f6 }];
31 set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { f5 }];
32 set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { f4 }];
33 set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { f3 }];
34 set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { f2 }];
35 set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { f1 }];
36 set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { f0 }];
```

Figura 9. Captura de pantalla las líneas del archivo de restricciones correspondientes a los puertos de salida *f6*, *f5*, *f4*, *f3*, *f2*, *f1*, y *f0* del circuito *funciones.vhd*.

Con esto quedan especificados todos los puertos del diseño respecto a los pines de la FPGA.

Es importante tener en cuenta que, si en algún momento cambiamos la FPGA para implementar el diseño, este archivo de restricciones no será válido, teniendo que seleccionar la plantilla de restricciones adecuada de partida.

4.- Síntesis, implementación y generación del Bitstream

El proceso de diseño e implementación de circuitos digitales usando la tecnología basada en FPGA-VHDL ha constado, hasta el momento, de los siguientes pasos:

1. Descripción del circuito en VHDL.
2. Generación del *test bench* y simulación.
3. Asignación de pines de la FPGA para los puertos de entrada y salida del diseño.

Con esto se tiene completamente definido y depurado (si la simulación es correcta) el diseño. Es el momento de que el entorno de desarrollo Vivado implemente el diseño especificado con los códigos VHDL y con las restricciones de usuario especificadas en el XDC. Con esta información el entorno de desarrollo realiza los siguientes pasos:

1. **Síntesis**. Este proceso convierte la descripción VHDL de un circuito en puertas lógicas y componentes lógicos básicos. La síntesis viene a convertir el código VHDL en una *netlist*.
2. **Implementación**. La implementación recoge la netlist creada en el proceso de síntesis para realizar tareas tales como optimización, mapeo en los componentes lógicos de la FPGA (dentro de los CLB), ubicación de la lógica en los CLB de la FPGA y enrutamiento (ver Figura 1 de la Actividad 0).

3. **Generación del Bitstream**. En este proceso se genera un archivo binario de extensión *.bit* con toda la información necesaria para configurar la FPGA.

El acceso a estos procesos se encuentra en el panel *Flow Navigator* del entorno de desarrollo (ventana de la izquierda). Cada etapa descrita se puede realizar paso a paso. También se puede realizar todos los procesos secuencialmente haciendo clic en el último, esto es, si pulsamos sobre *Generate Bitstream* realiza antes la síntesis y la implementación previamente. (Figura 10).

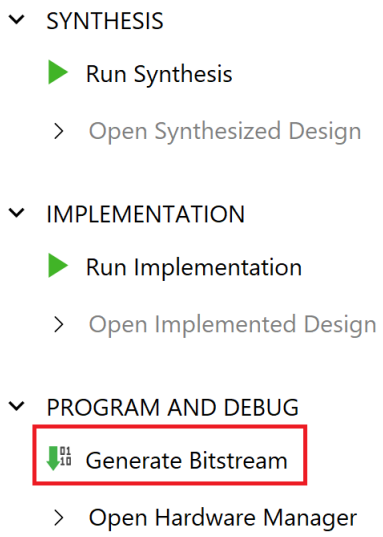

Figura 10. Captura de pantalla de la parte inferior del panel Flow Navigator con el comando *Generate Bitstream* resaltado en rojo.

En este caso, aparece una ventana emergente indicando que debe realizarse los pasos previos a *Generate Bitstream* (esto es, la síntesis y la implementación). Pulsar en *Yes* (Figura 11).

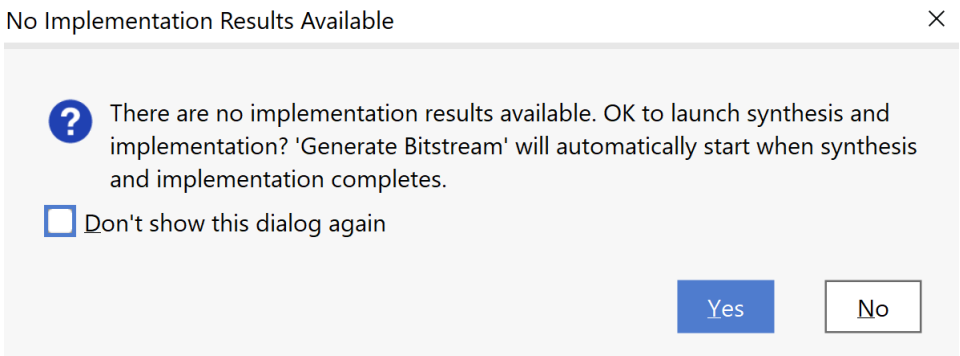

Figura 11. Captura de pantalla del mensaje emergente al solicitar Generate Bitstream sin tener realizados los procesos previos.

Por último, aparece un mensaje en la que podemos seleccionar el número de tareas. Esto depende del microprocesador del ordenador. Se dejan las opciones por defecto y se pulsa en *OK* para empezar.

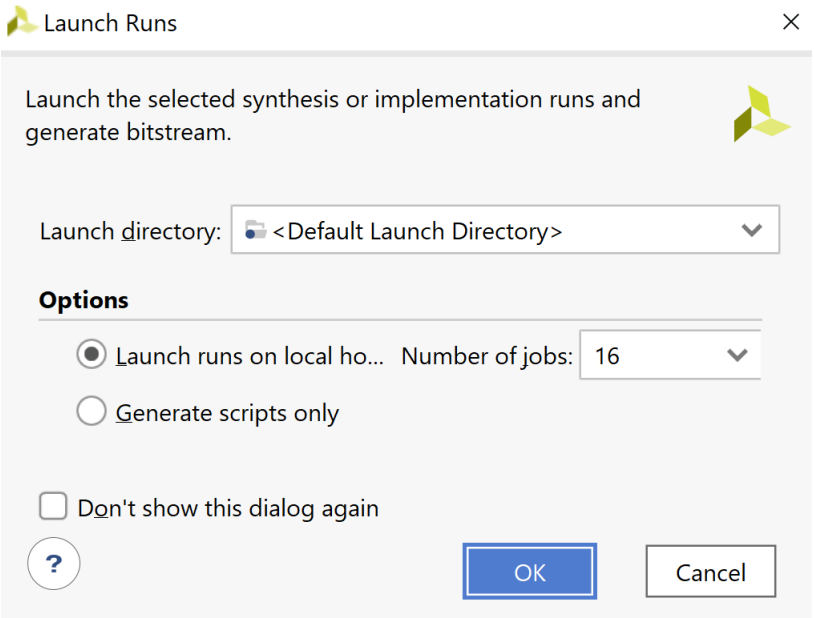

Figura 12. Captura de pantalla de *Launch Runs*.

Los procesos de síntesis, implementación y generación del bitstream son complejos. El entorno de desarrollo emplea unos pocos minutos en su cómputo, dependiendo de la complejidad del diseño y de las características del ordenador en el que Vivado se ejecuta.

NOTA: El proceso *Generate Bitstream* se encuentra en la categoría PROGRAM AND DEBUG (Figura 10). En realidad, el uso de PROGRAM no es afortunado, ya que está relacionado más con software que con hardware. Sería más adecuado emplear el término CONFIGURATION o CONFIGURATE, por ejemplo.

5.- Configuración de la FPGA con el diseño de la Actividad 1

Una vez finalizados los procesos de síntesis, implementación y generación del bitstream, aparece una nueva ventana emergente en la se debe seleccionar *Open Hardware Manager*, para poder conectar la tarjeta de desarrollo Nexys A7 con el ordenador y establecer la comunicación a través del puerto JTAG (Figura 13). El Hardware Manager se puede abrir en cualquier momento con el comando que se encuentra debajo de *Generate Bitstream* en el panel *Flow Navigator* (Figura 10).

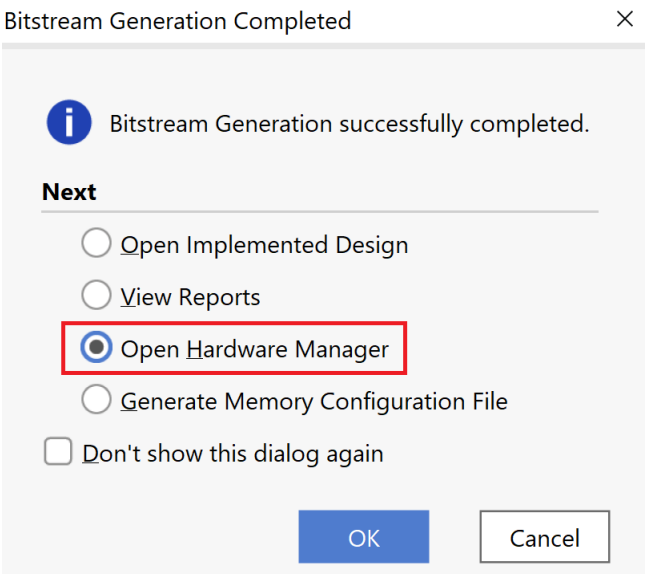

Figura 13. Captura de pantalla con Open Hardware Manager resaltado en rojo.

CONSIDERACIONES:

1. Para realizar este apartado de configuración de la FPGA es necesario disponer físicamente de la tarjeta de desarrollo y conectarla al PC.
2. La tarjeta no se puede configurar usando Vivado en Escritorio Virtual (Apéndice B de la Actividad 0).

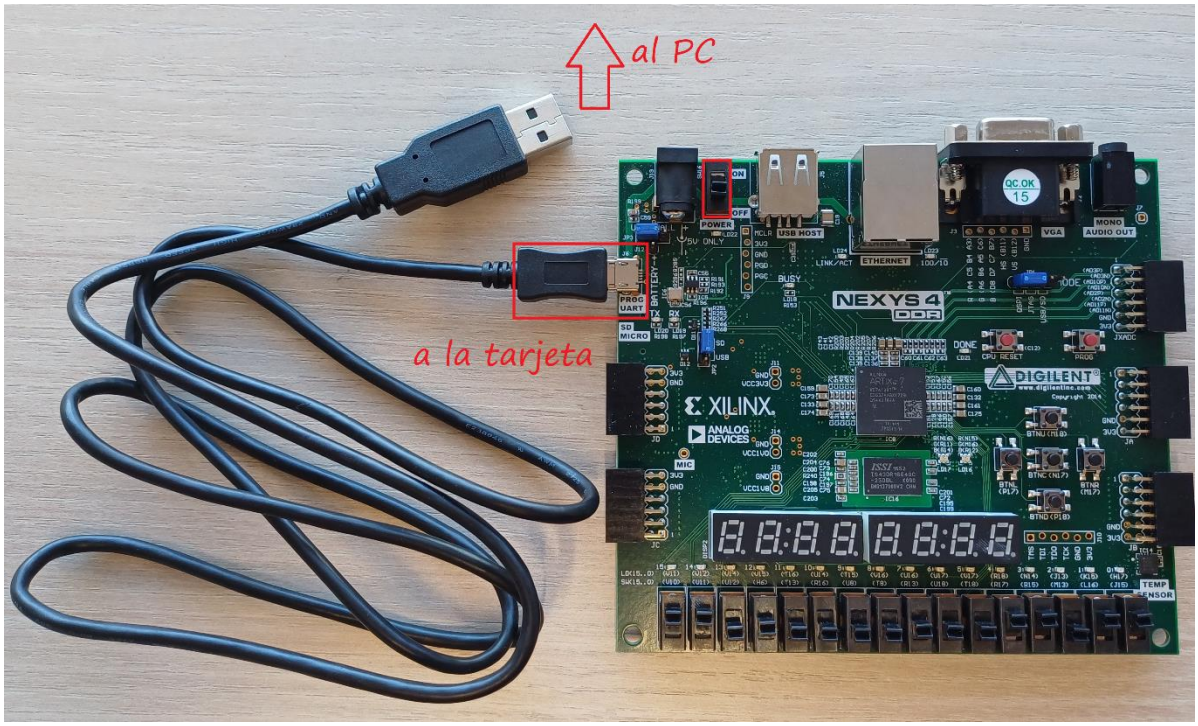

Figura 14. Foto de la tarjeta de desarrollo con el cable.

Para conectar adecuadamente la tarjeta de desarrollo Nexys A7 al PC hay que proceder de la siguiente manera:

1. Conectar el extremo del cable con el conector micro-USB a la tarjeta según se aprecia en la Figura 14.
2. Contectar el extremo del cable con el conector USB a cualquier puerto USB del PC.
3. Encender la tarjeta de desarrollo Nexys A7 conmutando el interruptor marcado en rojo en la Figura 14 de la posición OFF a ON.

Después de conectar la tarjeta al PC como se ha indicado, se tiene que establecer la comunicación y descargar en la tarjeta el fichero bitstream de extensión .BIT para configurar la tarjeta. Se puede apreciar que, inicialmente, aparece el mensaje *unconnected* junto al nombre de HARDWARE MANAGER, y debajo el mensaje *No hardware target is open*. Se debe proceder así:

1. En el Hardware Manager, hacer clic en *Open target* (Figura 15).
2. En el menú emergente, pulsar en *Auto Connect* (Figura 15).

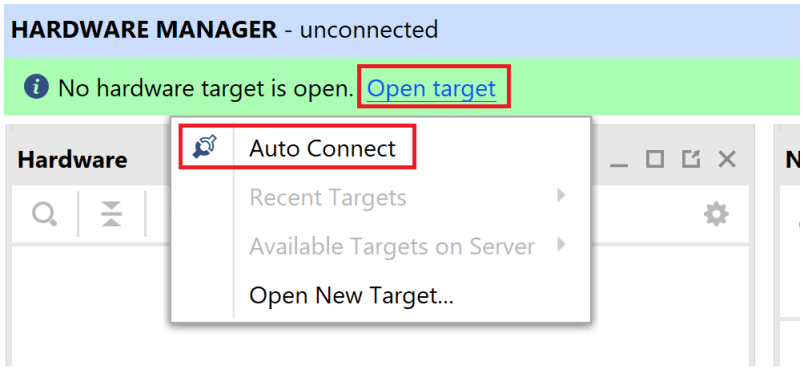

Figura 15. Captura de pantalla del *Hardware Manager* con *Open target* resaltado en rojo.

3. Después de pocos segundos, si la tarjeta de desarrollo está encendida y el cable no está roto, el entorno de desarrollo identifica la FPGA de la tarjeta. Sólo resta pulsar en *Program Device* (Figura 16).
4. En la ventana emergente, se comprueba que el fichero de bitstream es el correcto (*funciones.bit*) y pulsamos en *Program* (Figura 17). Después de unos escasos segundos, la FPGA queda configurada.

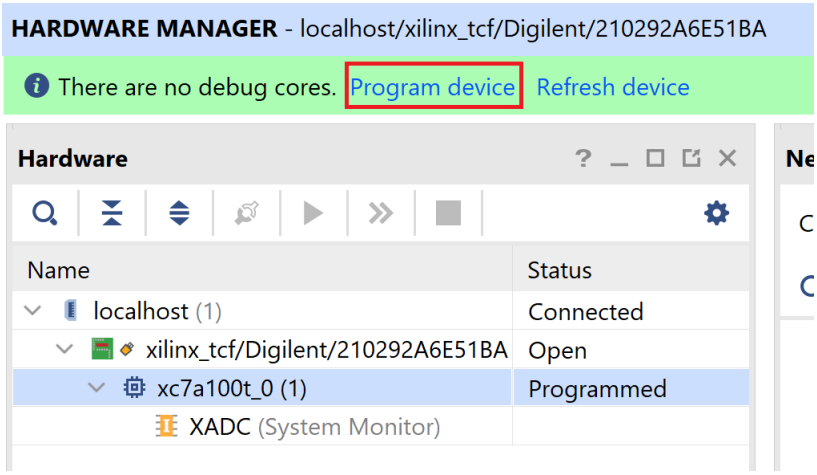

Figura 16. Captura de pantalla del *Hardware Manager* con *Program Device* resaltado en rojo.

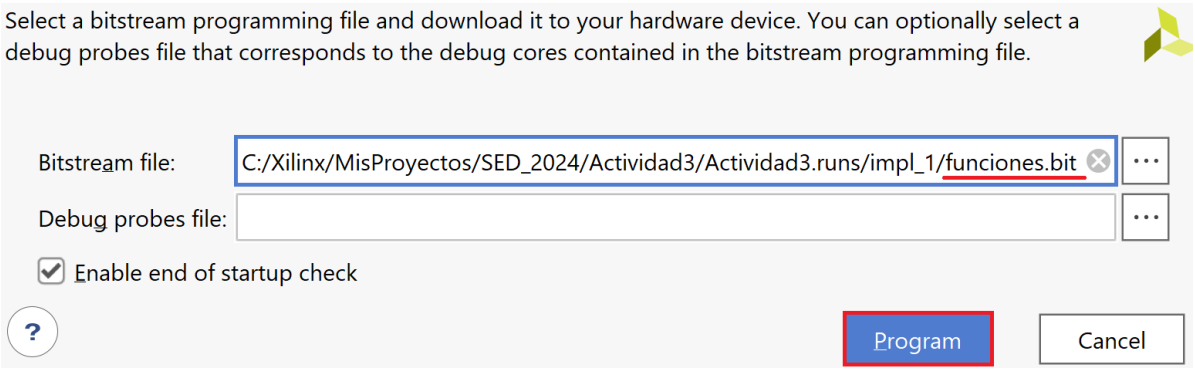

Figura 17. Captura de pantalla de selección del bitstream.

Una vez configurada la FPGA de la tarjeta, se puede interactuar con el circuito para comprobar su funcionamiento, cotejando con la tabla de verdad de partida del diseño (Tabla 2). En este caso se ha resaltado la combinación *a=0* y *b=1*. La tarjeta configurada se muestra en la Figura 18. En este caso, se han conmutado los interruptores para ingresar en el circuito los valores de a=0 y b=1. Puede comprobarse que los leds de salida de las funciones coinciden con los valores de la Tabla 2 (led apagado, función igual a 0, led encendido, función igual a 1). Para comprobar la correcta implementación del circuito, hay que comprobar todos los valores de la Tabla 2.

a	b	f_0	f_1	f_2	f_3	f_4	f_5	f_6
0	0	1	0	1	0	1	0	1
0	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**0**
1	0	0	0	1	1	0	1	0
1	1	0	1	0	1	0	0	1

Tabla 2. Tabla de verdad de las funciones lógicas básicas del circuito descrito en *funciones.vhd*

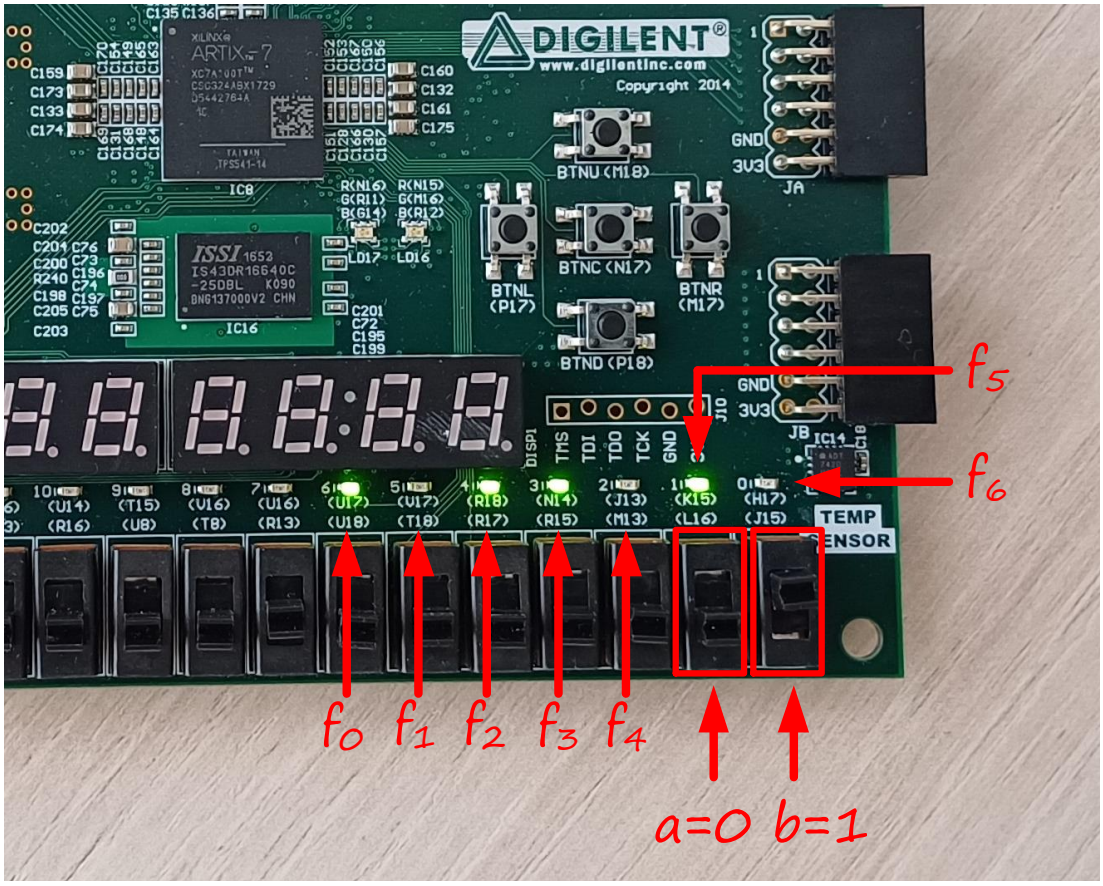

Figura 18. Tarjeta configurada con el circuito *funciones*.

Para cerrar el Hardware Manager debe hacerse clic en el aspa que aparece en la barra azul a la derecha (marcada en rojo en la Figura 19). Con esto se vuelve a la vista principal de Vivado.

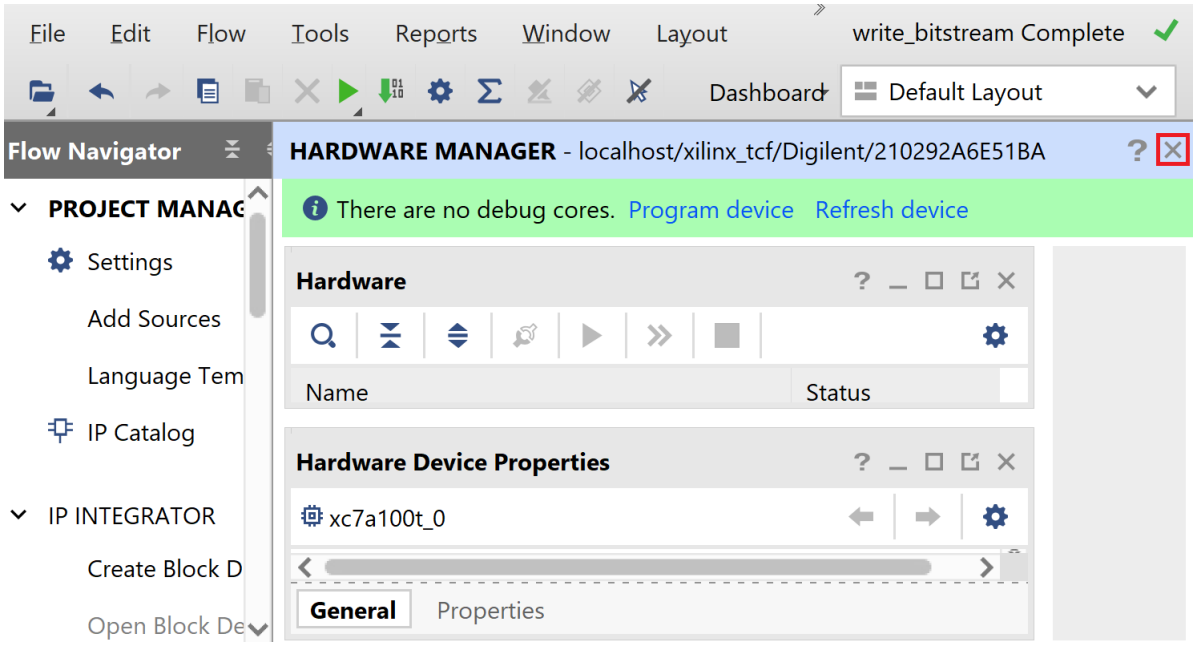

Figura 19. Captura del Hardware Manager con el aspa *Close Hardware Manager* resaltada en rojo.

6.- Configuración de la FPGA con el diseño de la Actividad 2

Para configurar la FPGA con el circuito descrito en el código *funciones2.vhd* en la Actividad1 se debe proceder de la siguiente manera:

1. Seleccionar *funciones2.vhd* como top module en la ventana de jerarquía. Para ello se hace uso del comando *Set as Top* haciendo clic con el botón derecho en *funciones2.vhd*. Realizando esta operación, *funciones2* aparece en negrita y con el icono de *top level* a su izquierda (Figura 20).

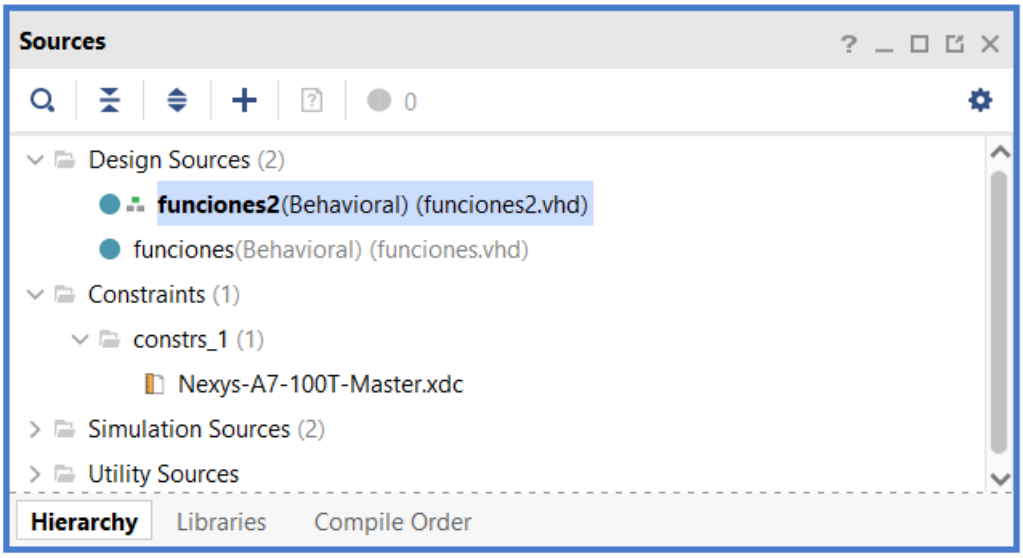

Figura 20. Captura de pantalla de selección del bitstream.

2. La interfaz del nuevo circuito es distinta, ya que consta de 4 puertos de entrada *a*, *b*, *c*, *d*; y 8 puertos de salida *g0*, *g1*, *g2*, *g3*, *g4*, *g5*, *g6*, *g7*. Por tanto, hay que modificar el archivo de asignaciones de pines de manera consecuente. Se asigna 4 *switches* para las entradas y 8 leds para las salidas del circuito en el fichero *Nexys-A7-100T-Master.xdc* según se muestra en la Figura 21.
3. Tras realizar estas actualizaciones, hay que volver a realizar los procesos de síntesis, implementación, generación del bitstream; así como volver a configurar la FPGA de la tarjeta de desarrollo, esta vez con el nuevo archivo *funciones2.bit*. Básicamente, hay que repetir las acciones descritas en el Apartado 4 y en el Apartado 5.

Nuevamente hay que comprobar que el circuito queda configurado correctamente. En este caso, se contrasta con la simulación realizada en la Actividad 2, que se muestra en la Figura 22.

```
11 ##Switches
12 set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { d }];
13 set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { c }];
14 set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { b }];
15 set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { a }];

28
29 ## LEDs
30 set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { g7 }];
31 set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { g6 }];
32 set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { g5 }];
33 set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { g4 }];
34 set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { g3 }];
35 set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { g2 }];
36 set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { g1 }];
37 set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { g0 }];
```

Figura 21. Captura de pantalla las líneas del archivo de restricciones *Nexys-A7-100T-Master.xdc* correspondientes a los puertos del circuito descrito en el fichero *funciones2.vhd*.

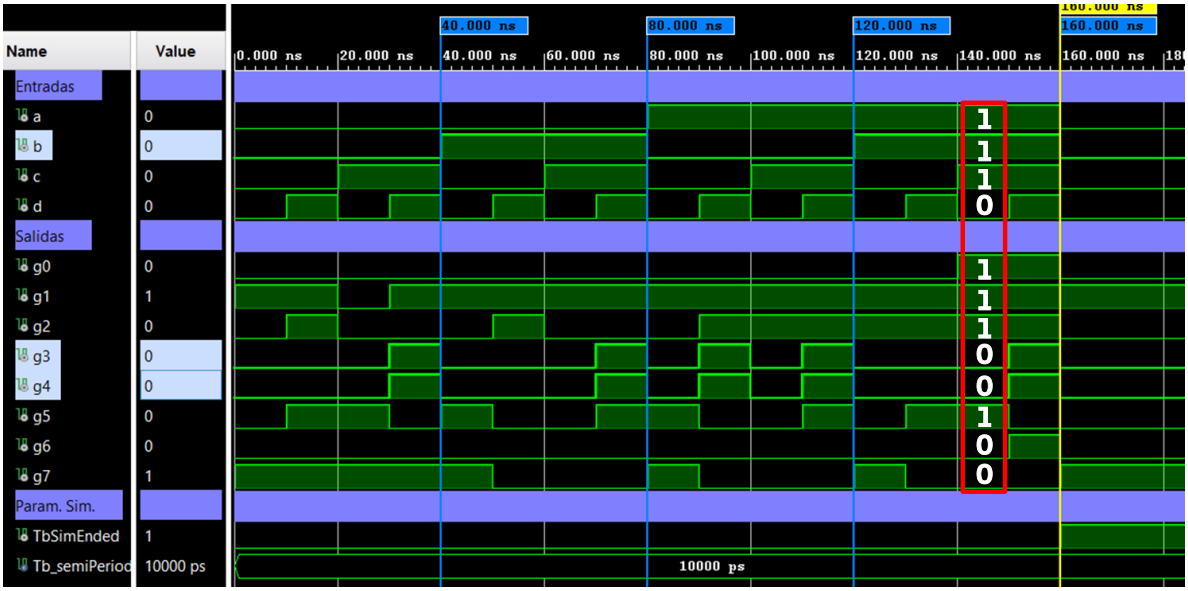

Figura 22. Captura del cronograma de simulación de *funciones2*.

En la figura 23 se muestra un detalle de la tarjeta de desarrollo con el circuito *funciones2* configurado en la FPGA de dicha tarjeta. En este caso, la imagen muestra los resultados de ingresar a través de los interruptores de entrada los valores a=1, b=1, c=1 y d=0. Se comprueba que los leds de salida asociados a las funciones g0, g1, g2, g3, g4, g5, g6 y g7 coinciden con los valores resaltados en rojo del cronograma de la Figura 22. Es necesario comprobar todas las combinaciones de entrada para validar el circuito que se ha configurado en la FPGA.

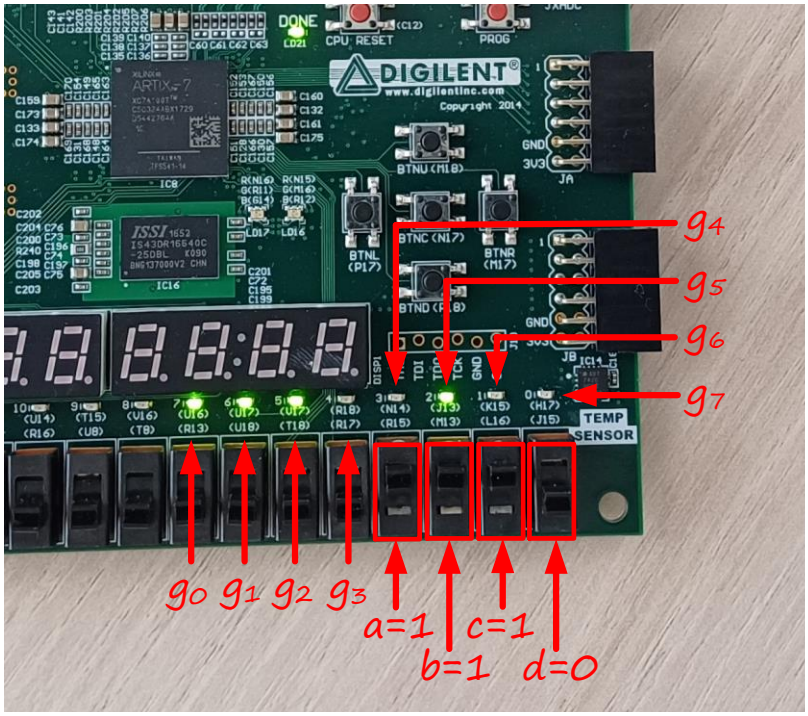

Figura 23. Tarjeta configurada con el circuito *funciones2*.

Por último, para desconectar la tarjeta de desarrollo, se procede de la siguiente manera:

1. Cerrar el *Hardware Manager*.
2. Apagar la tarjeta de desarrollo. Poner el interruptor de encendido en posición de OFF (Figura 14).
3. Desconectar los cables del PC y la tarjeta de desarrollo.

Referencias

[1] https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual
[2] https://github.com/Digilent/digilent-xdc/blob/master/Nexys-A7-100T-Master.xdc